

Технологии
Программирования



Манипулируя
энергией вселенной

Магия

Программирование — процесс и искусство создания компьютерных программ с помощью языков программирования^{[1][2]}.



- Программирование сочетает в себе элементы искусства, науки, математики и инженерии.
- В узком смысле слова, программирование рассматривается как *кодирование* — реализация одного или нескольких взаимосвязанных алгоритмов на некотором языке программирования. В более широком смысле, программирование — *процесс создания программ*, то есть разработка программного обеспечения.
- Большая часть работы программиста связана с написанием исходного кода на одном из языков программирования.
- Различные *языки программирования* поддерживают различные стили программирования (т. н. парадигмы программирования). Отчасти, искусство программирования состоит в том, чтобы выбрать один из языков, наиболее полно подходящий для решения имеющейся задачи. Разные языки требуют от программиста различного уровня внимания к деталям при реализации алгоритма, результатом чего часто бывает компромисс между простотой и производительностью (или между временем программиста и временем пользователя).
- Единственный язык, напрямую выполняемый процессором — это *машинный язык* (также называемый машинным кодом). Изначально, все программисты прорабатывали каждую мелочь в машинном коде, но сейчас эта трудная работа уже не делается. Вместо этого, программисты пишут исходный код, и компьютер (используя компилятор, интерпретатор или ассемблер) транслирует его, в один или несколько этапов, уточняя все детали, в машинный код, готовый к исполнению на целевом процессоре. Даже если требуется полный низкоуровневый контроль над системой, программисты пишут на языке ассемблера, мнемонические инструкции которого преобразуются один к одному в соответствующие инструкции машинного языка целевого процессора.

Бла-бла-бла... Программирование - превращение одних данных в другие

Энергии

Имя - существительное

- Имя должно быть кратким и содержать информацию о том, что храниться в переменной

```
uint16_t m_color_rgb565; //переменная содержит 3х компонентный цвет размером 2 байта в раскладке r5, g6, b5
```

Типы

- Использовать максимально говорящие типы, минимально необходимого размера.

```
uint8_t m_drawing_context; //из типа видно, что контекстов не может быть больше 255
```

- Для сложных типов предпочитать структуры, стараться минимизировать использование анонимизированных таплов или пар.

RAII (*Resource Acquisition Is Initialization*)

- Переменная объявляется непосредственно перед использованием:

**ВСЕГДА ИНИЦИАЛИЗИРУЙТЕ ПЕРЕМЕННЫЕ СРАЗУ: В
МОМЕНТ ОБЪЯВЛЕНИЯ**



Вопрос!

Как данные представлены в программе?

Переменные, константы

Простые типы, составные типы (структуры, массивы...)

Сотворение заклинания



Инициализация

- Неинициализированная

```
int Uninitialized; // значение переменной неизвестно. есть риск ошибок.
```

- Список инициализации {}

```
int Initialized = {4};
```

- Копирование '='

```
int Copied = 4;
```

- Конструирование ()

```
int Constructed(4);
```

Использование/Модифицирование

Удаление

Вопрос!

```
struct Object
{
    int    IntegerField;
    float FloatField;
};

Object my_object; // что будет в my_object.IntegerField?
```

А кто ж его знает...
RAII?

```
Object my_object = {1, 2.f};
```

А если она всегда должна создаваться с этими значениями?

А если полей много и все они разные, риск запутаться?

А если поля зависимы друг от друга?

Сложное заклинание

Конструирование

- По умолчанию. Работает с листами инициализации

```
struct Object
{
    int    IntegerField = 1;
    float FloatField   = 2.f;
};
```

- Constructor(ctor) - спец функция. Кастомизирует порядок конструирования объекта структуры и алгоритм инициализации полей.

```
struct Object
{
    int    IntegerField;
    float FloatField;
    Object(int custom_integer_value) // конструктор. Имя ctor то же что и у структуры. Retval отсутствует
    {
        IntegerField = custom_integer_value;
        FloatField = custom_integer_value * 2.f; // Внутри ctor'а может быть любая логика
    };
};
```



Сотворение сложного заклинания

```
struct Object
{
    int IntegerField = 1;
    float FloatField = 2.f;
};
Object my_object; // структура инициализирована сразу
```

```
struct Object
{
    int IntegerField;
    float FloatField;
    Object(int custom_integer_value)
    {
        IntegerField = custom_integer_value;
        FloatField = custom_integer_value * 2.f;
    };
};
Object my_object(1); // Если определен ctor, создание возможно только через конструирование
// Object my_object; приведет к ошибке компиляции
```



Неизменная

Const - “Переменная” не подлежащая изменению.

- Можно использовать для хранения константных абстракций.

```
const float Exponent = 2.718f;
```

- Помогает улучшить читаемость написанного кода.

```
float exponential_square = pow(Exponent, 2);
```

- Типизирует математические операции в соответствии со своим типом.

Избегайте использование magic numbers в коде



Вопрос!

Время жизни данных?

От определения, до }

Наше измерение.

```
#include <string>
int GlobalCounter = 0;
```



- Создание вне всех функций,
- Доступна в течение всего времени работы программы,
- Доступ до нее из других файлов через extern,

ЗА

- + Время жизни превосходит время работы кода,
- + Доступ “отовсюду”.

ПРОТИВ

- - Порядок инициализации глобальных переменных из разных файлов неопределен,
- - Нет контроля над временем жизни переменной,
- - Конфликт со статическими переменными,
- - Singleton - расширение кода основанного на глобальных переменных затруднительно,

**Использование глобальных переменных в коде считается плохим тоном.
Казалось бы причем тут RAI.**

Зеркальное измерение.



Глобальная статическая переменная

- static,
- Глобальная переменная доступная в рамках одного модуля компиляции (CPP файла),
- Extern не работает,
- Инициализируется ПЕРЕД глобальными переменными.

```
#include <string>
static int LocalCounter = 0;
```

Локальная статическая переменная

- Глобальная переменная доступная только внутри функции где она объявлена,
- Инициализируется при первом вызове функции.

```
void IncrementCounter()
{
    static int counter = 0; // выполнится только при первом вызове функции
    printf("Counter value %d\n", counter++);
}
```

Вопрос!

```
struct Object
{
    static int    StaticIntField;
    static float StaticFloatField;
    float FloatField;
};
```

Статическое поле структуры

- Глобальная переменная в рамках структуры,
- Инициализируется сразу,
- Доступна отовсюду где видна структура (extern не нужен),
- Не требует создание объекта для использования,
- Не является частью структуры.

```
Object::StaticIntField = 2;
```



Ну всё, пока-пока